



杭州电子科技大学

操作系统内核设计

Unfound OS

董洁 李嫣然 赵果利

2026 年 1 月

目录

第 1 章 摘要	1
1.1 Unfound 内核开发背景	1
1.2 Unfound 整体内核设计	1
第 2 章 系统概述	1
2.1 Unfound 整体架构概述	1
2.2 Unfound 代码结构	3
第 3 章 内存管理模块	3
3.1 内存管理模块框架概述	3
3.2 多分配器框架实现	4
3.3 分配器测试框架设计实现	6
第 4 章 文件系统模块	9
4.1 虚拟文件系统架构设计	9
4.2 具体实现	10
4.3 文件系统总结	17
第 5 章 进程管理模块	17
5.1 进程管理模块框架概述	17
5.2 读写锁分离与并发架构实现	17
5.3 进程生命周期与资源隔离实现	19
5.4 进程间通信 (IPC) 系统实现	21
5.5 进程管理模块总结	24
第 6 章 总结与展望	24
6.1 项目总结	24
6.2 核心价值	25
6.3 现存局限	25
第 7 章 致谢	27

第 1 章 摘要

1.1 Unfound 内核开发背景

本项目基于 ArceOS 开展开发工作。ArceOS 是一个采用 Rust 语言编写的实验性操作系统，最初以 Unikernel 架构实现，旨在通过内存安全与最小化攻击面提升系统可靠性。其核心设计以组件库 (library-based) 的形式，为操作系统内核提供基础设施与核心功能，所有模块均以独立的 Rust crate 组织，支持按需组合与灵活替换。

ArceOS 的架构高度模块化，涵盖以下关键组件：

- 硬件与资源抽象层：包括 axhal (硬件抽象层)、axalloc (安全内存分配器) 和 axdriver (统一驱动框架)，为跨平台支持奠定基础；
- 并发与任务管理：通过 axtask (轻量级任务调度) 和 axsync (异步同步原语)，原生支持高效异步与并发编程；
- 文件系统：基于 axfs 提供虚拟文件系统 (VFS) 抽象；
- 系统服务：如 axlog (日志系统)、arceos::posix_api (部分 POSIX 兼容接口)，便于移植传统用户态应用。

本项目基于 ArceOS 现有框架，旨在通过深度开发推动系统功能向完整操作系统演进。在初赛阶段，我们聚焦于三大核心模块——内存管理、进程管理、文件系统，进行了系统性优化，显著提升了内核的稳定性与功能性，并大幅增强了对通用用户态程序的支持能力。

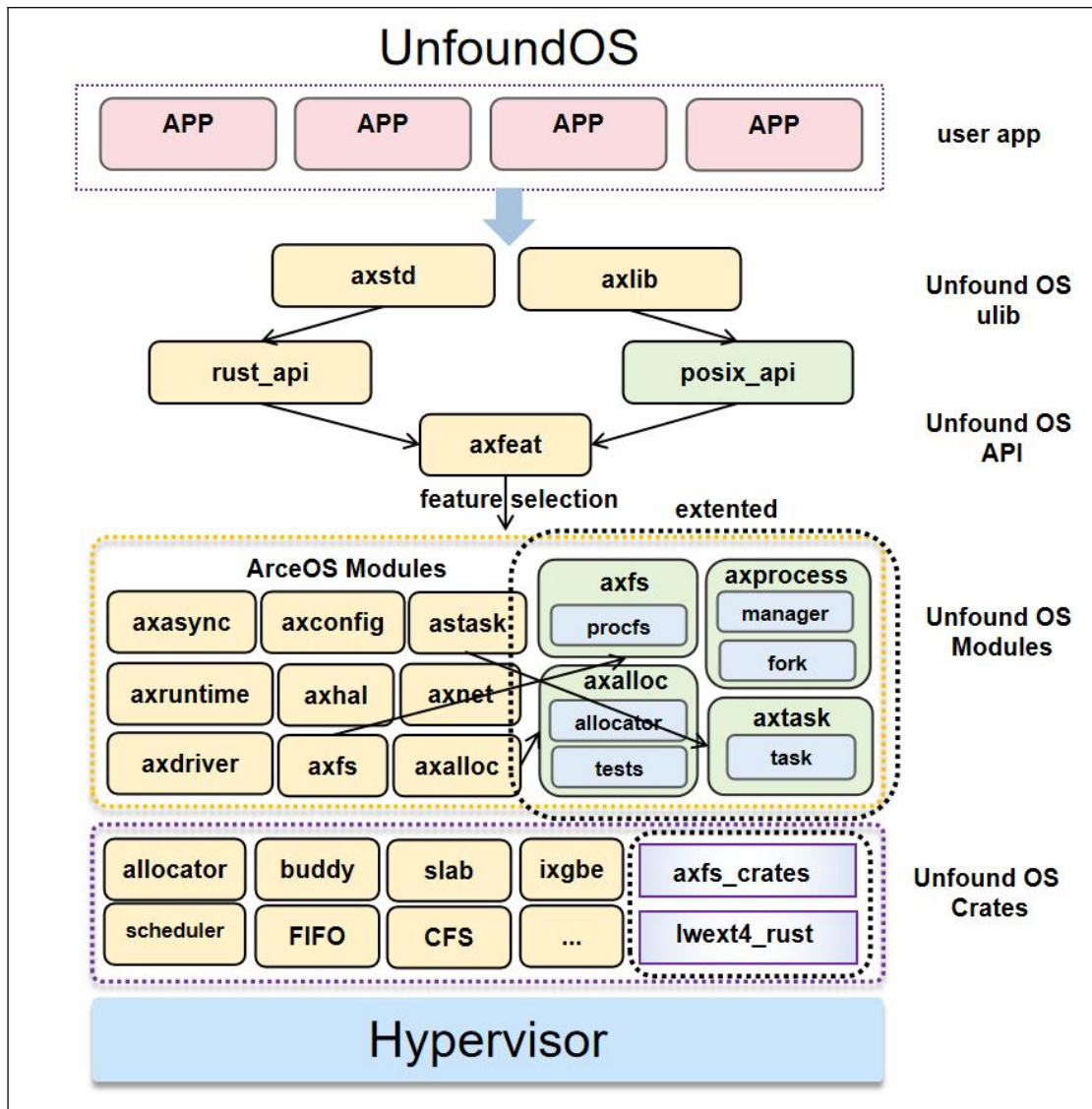
1.2 Unfound 整体内核设计

本项目的总体架构延续并深化了 ArceOS 的组件化设计理念，采用高度模块化的组织方式，将内核功能拆解为多个职责清晰、接口明确的独立模块，每个模块负责特定的功能。这样的设计使得内核更加灵活，易于扩展和维护，同时模块可以从内核中拆分出来，作为组件库提供给其他项目。

第 2 章 系统概述

2.1 Unfound 整体架构概述

内核的整体架构如下图所示：



在三大模块的创新主要体现在以下方面：

- 内存管理：在 UnfoundOS 不仅实现了 Buddy、Bitmap 和 Hybrid 三种物理内存分配算法，更打造了一套全维度测试框架，支持在用户态与内核态双环境下对吞吐量、碎片率、稳定性等指标进行量化对比；该模块深度集成于 ArceOS 的模块化架构，依托 Rust 的内存安全特性，既为内核提供高效可靠的物理页管理，又成为操作系统教学中连接理论与工程实践的实验平台。
- 进程管理：本次改进的核心创新在于将现代操作系统的多进程管理能力引入轻量级 Unikernel 架构。RwLock 优化、生命周期闭环、资源隔离与协作（虚拟化与 IPC）同时补全了管道、信号、消息队列三大 IPC 子系统，赋予了 UnfoundOS 运行 Shell、Web 服务器等复杂多进程协作应用的能力。
- 文件系统：引入统一的 FileLike I/O 抽象模型，完整实现 POSIX *at 系列系统调用（如 openat、mkdirat、unlinkat），实现了任意路径动态挂载以及采用最长前缀匹配机制实现嵌套挂载，增强了文件系统的功能性和可扩展性。

三大模块协同演进，共同推动 UnfoundOS 从一个实验性 Unikernel 框架，成功转型为具备完整宏内核能力、可运行真实应用的操作系统原型。

2.2 Unfound 代码结构

- >modules: 内核组件 (内存、调度、驱动、网络、文件系统、日志等)
 - >axalloc: 内存分配器框架与测试模块
 - >axfs: 文件系统框架与测试
- >crates: crates 组件 (新增 axfs_crates、lwext4_rust)
- >api: 应用接口 (axfeat、arceos_api、arceos_posix_api)
- >ulib: 用户态库 (axstd、axlibc)
- >examples: 子系统示例与冒烟测试
- >configs: 平台配置
- >scripts、tools: 构建与 QEMU 运行脚本
- >doc: 设计文档与图表

第 3 章 内存管理模块

3.1 内存管理模块框架概述

UnfoundOS 的内存管理模块深度集成于 ArceOS 的模块化内核框架, 不仅覆盖通用内核开发的核心技能, 更充分体现了 ArceOS “以 crate 为单元、按需组合、安全优先” 的设计哲学。该模块不仅是一个功能完备的内核级物理内存分配器, 更是一个集算法验证、工程实践与系统思维培养于一体的实验教学平台。基于 Rust 语言的内存安全抽象, 模块实现了 Buddy (伙伴系统)、Bitmap (位图管理) 和 Hybrid (混合策略) 三种分配算法, 并配套一套全维度测试框架, 将抽象的内存管理理论转化为可观察、可对比、可复现的实验现象, 有效弥合理论教学与工程实践之间的鸿沟。

3.2 多分配器框架实现

多分 `ageAllocator` trait 抽象统一接口，以此支持 Buddy、Bitmap、Hybrid 等多种分配策略的无缝切换；该框架具备三大关键特性，即支持运行时动态选择分配器而无需重新编译内核、允许分配器的并行对比测试以便利性能评估与迁移验证、实现统一的内存池管理与生命周期控制。

核心代码：

```
01  /// Minimal allocator trait for page-level operations
02  used by the runtime
03  /// switching infrastructure.
04  pub trait PageAllocator: Send + Sync {
05      /// Return a short name of the allocator.
06      fn name(&self) -> &'static str;
07
08      /// Initialize allocator with a region starting at
09      `start_vaddr` of `size` bytes.
10      fn init(&self, start_vaddr: usize, size: usize) ->
11      Result<(), AllocError>;
12
13      /// Allocate contiguous pages: returns start
14      virtual address on success.
15      fn alloc_pages(&self, num_pages: usize,
16      align_pow2: usize) -> Result<usize, AllocError>;
17
18      /// Allocate contiguous pages at an exact start
19      address.
20      fn alloc_pages_at(
21          &self,
22          start: usize,
23          num_pages: usize,
24          align_pow2: usize,
25      ) -> Result<usize, AllocError>;
26
27      /// Deallocate contiguous pages starting from
28      `pos`.
29      fn dealloc_pages(&self, pos: usize, num_pages:
30      usize);
31
32      /// Optional: Return diagnostics (fragmentation and
33      free memory).
34      /// Default implementation returns (0.0, 0)
35      indicating no diagnostic data available.
```

```

    fn get_stats(&self) -> (f64, usize) {
        (0.0, 0)
    }
}

```

具体实现:

modules/axalloc/src/allocators/buddy.rs - Buddy 分配器

modules/axalloc/src/allocators/bitmap.rs - Bitmap 分配器

modules/axalloc/src/allocators/hybrid.rs - Hybrid 混合分配器

运行时切换机制:

```

0  #[cfg(feature = "runtime-switch")]
1  pub mod runtime {
0      use super::PageAllocator;
2      use allocator::AllocError;
0      use core::option::Option;
3      use kspin::SpinNoIrq;
0
4      // Global storage for the runtime-selected page
0  allocator
5      static GLOBAL_PAGE_ALLOC: SpinNoIrq<Option<Box<dyn
0  PageAllocator>>> =
6          SpinNoIrq::new(None);
0
7      /// Try to set the global runtime allocator. Overwrites
0  any previous value.
8      pub fn set_runtime_allocator(a: Box<dyn
0  PageAllocator>) {
9          let mut slot = GLOBAL_PAGE_ALLOC.lock();
1         *slot = Some(a);
0     }
1
1     /// Helper to create an allocator by name.
1     pub fn make_by_name(name: &str) -> Result<Box<dyn
2  PageAllocator>, &'static str> {
1         match name {
3             "buddy" => {
1                 #[cfg(feature = "buddy")]
4                 { Ok(Box::new(crate::allocators::BuddyA
1  llocator::new())) }
5                 #[cfg(not(feature = "buddy"))]
1                 { Err("buddy feature not enabled") }
6             }
        }
    }
}

```

```

1         "bitmap" => {
7             #[cfg(feature = "bitmap")]
1             { Ok(Box::new(crate::allocators::Bitmap
8 Allocator::new())) }
1             #[cfg(not(feature = "bitmap"))]
9             { Err("bitmap feature not enabled") }
2         }
0         "hybrid" => {
2             #[cfg(feature = "hybrid")]
1             { Ok(Box::new(crate::allocators::Hybrid
2 Allocator::new())) }
2             #[cfg(not(feature = "hybrid"))]
2             { Err("hybrid feature not enabled") }
3         }
2         _ => Err("unknown allocator name"),
4     }
2 }
8 }

```

3.3 分配器测试框架设计实现

UnfoundOS 分配器测试框架具备模块化、可扩展的测试能力，专门用于评估内存分配器的性能、碎片情况、稳定性及时间维度表现，其核心支持涵盖多方面测试需求，包括基础性能测试（分配成功率、分配与释放耗时、吞吐量）、碎片测试（外部与内部碎片率、元数据开销）、稳定性测试（分配失败率、内存泄漏检测、多线程竞争开销）、时间维度测试（1 小时、6 小时、24 小时的长期运行模拟）、运行时切换测试（分配器切换耗时与兼容性），以及内核兼容测试（适配嵌入式与内核环境的 no_std 测试能力）。

基础测试:

```

01 /// Run basic performance tests on an allocator.
02 ///
03 /// This measures:
04 /// - Allocation success rate
05 /// - Average allocation/deallocation time
06 /// - Throughput (ops/sec)
07 /// - Time distribution (min, max, p50, p99)
08 pub fn run_basic_performance_test<A: PageAllocator
09 + ?Sized>(
10     allocator: &A,
11     config: &BasicPerfConfig,
12 ) -> BasicPerformanceMetrics {
13     let mut metrics = BasicPerformanceMetrics::new();
14     let mut alloc_times: Vec<u64> = Vec::new();

```



```

14     let mut dealloc_times: Vec<u64> = Vec::new();
15     let mut allocated: Vec<(usize, usize)> = Vec::new();
16     // (addr, size_pages)
17
18     let mut rng = SimpleRng::new(42);
19     let test_start = Instant::now();
20
21     for _i in 0..config.num_allocations {
22         if !allocated.is_empty() && rng.next_f64() <
23 config.dealloc_during_alloc_ratio {
24             let idx = rng.next_usize(allocated.len());
25             let (addr, size) =
26 allocated.swap_remove(idx);
27             let start = Instant::now();
28             allocator.dealloc_pages(addr, size);
29             let elapsed = start.elapsed().as_nanos() as
30 u64;
31             dealloc_times.push(elapsed);
32         }
33
34         let size_idx =
35 rng.next_usize(config.allocation_sizes.len());
36         let size = config.allocation_sizes[size_idx];
37
38         let start = Instant::now();
39         let result = allocator.alloc_pages(size,
40 config.alignment);
41         let elapsed = start.elapsed().as_nanos() as
42 u64;
43
44         metrics.total_allocations += 1;
45
46         match result {
47             Ok(addr) => {
48                 metrics.successful_allocations += 1;
49                 allocated.push((addr, size));
50                 alloc_times.push(elapsed);
51             }
52             Err(_) => {
53                 metrics.failed_allocations += 1;
54             }
55         }
56     }

```

```

    metrics.calculate_success_rate();
    metrics
}

```

碎片测试:

```

01 /// Collect a fragmentation snapshot from allocator
02 stats.
03 fn collect_fragmentation_snapshot<A: PageAllocator
04 + ?Sized>(
05     allocator: &A,
06     time_hours: f64,
07     total_memory: usize,
08     tracked_requested: usize,
09     tracked_actual: usize,
10 ) -> FragmentationSnapshot {
11     let (frag_rate, total_free) =
12 allocator.get_stats();
13     let max_contiguous = if total_free > 0 {
14         ((1.0 - frag_rate) * total_free as f64) as usize
15     } else {
16         0
17     };
18     let actual_allocated = if total_memory > total_free
19 {
20         total_memory - total_free
21     } else {
22         0
23     };
24     let internal_waste = if tracked_actual >
25 tracked_requested {
26         tracked_actual - tracked_requested
27     } else {
28         0
29     };
30     let mut snapshot = FragmentationSnapshot {
31         time_hours,
32         total_free_bytes: total_free,
33         max_contiguous_free_bytes: max_contiguous,
34         external_fragmentation_rate: frag_rate *
35 100.0,
36         total_allocated_bytes: actual_allocated,
37         internal_waste_bytes: internal_waste,
38         internal_fragmentation_rate: 0.0,

```

```

    };
    if tracked_actual > 0 {
        snapshot.internal_fragmentation_rate =
            (internal_waste as f64 / tracked_actual as f64) * 100.0;
    }
    snapshot
}

```

时间维度:

```

01 /// Run fragmentation tests over simulated time periods.
02 pub fn run_fragmentation_test<A: PageAllocator
03 + ?Sized>(
04     allocator: &A,
05     config: &FragmentationTestConfig,
06 ) -> FragmentationMetrics {
07     let mut metrics = FragmentationMetrics::new();
08     let mut rng = SimpleRng::new(42);
09     let test_start = Instant::now();
10     for _ in 0..config.num_iterations {
11         let size_idx =
12             rng.next_usize(config.allocation_sizes.len());
13         let size = config.allocation_sizes[size_idx];
14         if let Ok(addr) = allocator.alloc_pages(size,
15             config.alignment) {
16             allocator.dealloc_pages(addr, size);
17         }
18     }
19     metrics.total_time =
20         test_start.elapsed().as_secs_f64();
21     metrics
22 }

```

第 4 章 文件系统模块

4.1 虚拟文件系统架构设计

虚拟文件系统（Virtual File System, VFS）是现代操作系统中用于抽象和统一多种具体文件系统的核心机制。其核心思想是：将所有 I/O 请求首先路由至一个中间抽象层（即 VFS），由该层解析请求语义，并将其转发给底层具体的文件系统实现。这种设计使得上层应用无需关心底层存储格式，从而实现“一次编

写，多后端运行”的目标。ArceOS 的虚拟文件系统（VFS）作为 I/O 子系统的基石，虽已提供基本的文件读写能力，并遵循统一抽象、灵活扩展与透明访问的设计原则，但其功能仍存在明显局限：缺乏对目录操作（如创建、遍历和删除）的完整支持，未实现 POSIX 标准中的关键 *at 系列系统调用（如 openat、mkdirat、unlinkat）。

针对 ArceOS 虚拟文件系统（VFS）在目录操作、POSIX 兼容性及扩展能力等方面的不足，Unfound 提出了系统性改进，实现了以下关键创新：

- 统一 I/O 抽象模型
- 完整 POSIX 目录支持
- 动态多后端挂载机制：支持在任意路径（如 /mnt/usb）运行时挂载 FAT、EXT4 等文件系统，通过最长前缀匹配实现智能请求路由；

4.2 具体实现

1.统一 I/O 抽象模型：与 ArceOS 一脉相承，Unfound OS 同样以 Rust 的 trait 机制为核心，对虚拟文件系统（VFS）进行了结构化抽象。我们在原有基础上进一步扩展了两套关键接口，分别对应文件系统级和节点级操作。具体定义如下：

```
01  /// VFS 文件系统级操作接口（对应整个文件系统实例）
02
03  /// 要求实现线程安全（Send + Sync），支持并发访问
04
05  pub trait VfsOps: Send + Sync {
06
07      /// 在指定路径挂载一个 VFS 节点（如将 FAT 分区挂载到
08  /mnt/usb)
09
10      fn mount(&self, path: &str, mount_point:
11  VfsNodeRef) -> VfsResult;
12
13
14      /// 卸载当前文件系统
15
16      fn umount(&self) -> VfsResult;
17
18
19      /// 格式化底层存储设备（如清空并初始化文件系统元数据）
20
21      fn format(&self) -> VfsResult;
22
23
24      /// 获取文件系统的统计信息（如总空间、可用空间等），用于
25  statfs 系统调用
26
27      fn statfs(&self, path: *const core::ffi::c_char,
```

```

26 info: *mut FileSystemInfo) -> VfsResult<usize>;
27
28     /// 返回该文件系统的根目录节点引用
29
30     fn root_dir(&self) -> VfsNodeRef;
31 }
32
33 /// VFS 节点级操作接口（对应单个文件、目录或设备节点）
34
35 /// 要求实现线程安全（Send + Sync）
36
37 pub trait VfsNodeOps: Send + Sync {
38     /// 打开节点时的回调（通常用于设备或特殊文件，普通文件可留
39     空）
40
41     fn open(&self) -> VfsResult { Ok(()) }
42
43
44     /// 关闭或释放节点资源时的回调
45
46     fn release(&self) -> VfsResult { Ok(()) }
47
48     /// 获取节点的基本属性（类型、大小、权限、时间戳等）
49
50     fn get_attr(&self) -> VfsResult<VfsNodeAttr>;
51
52     /// 获取节点的扩展属性（支持更精细的元数据，如纳秒级时间戳）
53
54     fn get_attr_x(&self) -> VfsResult<VfsNodeAttrX>;
55
56     /// 从指定偏移量读取数据到缓冲区（用于普通文件）
57
58     fn read_at(&self, offset: u64, buf: &mut [u8]) ->
VfsResult<usize>;
59
60     /// 从指定偏移量写入数据（用于普通文件）
61
62     fn write_at(&self, offset: u64, buf: &[u8]) ->
VfsResult<usize>;
63
64     /// 读取目录项列表（用于目录节点，支持 ls、getdents 等）
65
66     fn read_dir(&self, start_idx: usize, dirents: &mut
[VfsDirEntry]) -> VfsResult<usize>;

```

```

        /// 在当前目录下查找子路径对应的节点（用于路径解析，如
lookup("foo/bar"))

        /// 使用 Arc<Self> 以支持目录遍历时的引用传递
        fn lookup(self: core::sync::Arc<Self>, path: &str)
        -> VfsResult<VfsNodeRef>;
    }

```

我们在 VfsNodeOps 中新增了 get_attr_x、set_atime、set_mtime、xattr（扩展属性）等接口，支持更丰富的元数据管理，便于兼容 POSIX/EXT4 等高级特性。

2. 完整 POSIX 目录支持:

Unfound OS 在 root.rs 中实现了符合 POSIX 语义的完整目录操作能力，涵盖路径解析与核心目录管理功能，包括:

- > 路径解析: 支持相对路径、绝对路径及../的规范化处理;
- > 节点生命周期管理: 提供 create（创建文件/目录）、lookup（查找节点）、remove（删除节点）、rename（重命名）等关键操作;
- > 当前工作目录（CWD）支持: 为用户态程序提供基于进程的当前目录上下文。相关目录操作函数，具体定义如下:

```

01  /// 在指定目录（或当前工作目录）下查找给定路径对应的 VFS 节点。
02  ///
03  ///
04  /// - `dir`: 可选的起始目录节点; 若为 None, 则从根目录开始解
05  析。
06  ///
07  /// - `path`: 要查找的相对或绝对路径（如 "a/b/c" 或
08  09  "/mnt/usb/file.txt"）。
10  ///
11  /// 返回查找到的节点引用。若路径以 '/' 结尾但目标不是目录，则
12  13  返回 `NotADirectory` 错误。
14  15  pub(crate) fn lookup(dir: Option<&VfsNodeRef>, path:
16  17  &str) -> AxResult<VfsNodeRef> {
18  19      if path.is_empty() {
19  20          return ax_err!(NotFound); // 空路径视为未找到
20  21      }
21  22

```

```

23 // 通过 parent_node_of 获取路径的父目录节点，然后在其下
24
25 lookup 最后一级组件
26
27 let node = parent_node_of(dir,
28 path).lookup(path)?;
29
30 // 如果路径以 '/' 结尾，但查找到的节点不是目录，则报错
31 if path.ends_with('/')
32 && !node.get_attr()?.is_dir() {
33     ax_err!(NotADirectory)
34 } else {
35     Ok(node)
36 }
37 }
38
39 /// 创建一个新目录。
40
41 ///
42 /// - `dir`: 可选的起始目录；若为 None，则从根目录开始解析路
43 径。
44
45 /// - `path`: 要创建的目录路径（如 "a/b/newdir"）。
46
47 ///
48 pub(crate) fn create_dir(dir: Option<&VfsNodeRef>,
49 path: &str) -> AxResult {
50     match lookup(dir, path) {
51         Ok(_) => ax_err!(AlreadyExists), // 目录已存在
52         Err(AxError::NotFound) => {
53             // 父目录存在但最后一级不存在，尝试创建
54             parent_node_of(dir, path).create(path,
55 VfsNodeType::Dir)
56         },
57         Err(e) => Err(e), // 其他错误（如权限、非目录中间
58 路径等）直接透传
59     }
60 }

```

```

    /// 删除一个空目录。
    ///
    /// - `dir`: 可选的起始目录;
    /// - `path`: 要删除的目录路径。
    ///
    pub(crate) fn remove_dir(dir: Option<&VfsNodeRef>,
    path: &str) -> AxResult {
        // validation + permission checks then delegate to
        parent.remove(path)
        // ...existing code...
    }

```

POSIX 系统调用绑定与上层映射

上述 VFS 能力通过 `arceos_posix_api` 模块向上暴露为标准系统调用接口。具体实现位于:

```

1 /home/hen/unfound/api/arceos_posix_api/src/imp/fs_compl
  ex_backup.rs

```

该文件负责将底层 VFS 操作 (如 `lookup`、`create_dir`、`remove_dir` 等) 映射到对应的 POSIX 系统调用 (如 `openat`、`mkdirat`、`unlinkat`、`renameat` 等), 并处理路径解析、权限检查、错误码转换等细节, 从而确保 GNU coreutils 等传统用户态程序能够正常运行。

3. 动态多后端挂载机制:

Unfound OS 的动态挂载能力在 `root.rs` 中实现, 核心包括:

- > 挂载表 (Mount Table): 以线程安全的方式维护所有已挂载文件系统的元信息;
- > 挂载/卸载操作: 支持在任意绝对路径 (如 `/mnt/usb`) 挂载新的文件系统, 并完成目录节点创建与后端初始化;
- > 最长前缀匹配 (Longest Prefix Match): 在路径解析时, 自动选择最具体的挂载点, 确保嵌套挂载 (如 `/mnt` 与 `/mnt/usb` 共存) 的正确路由。

示例 (`MountPoint` / `RootDirectory.mount` / `lookup_mounted_fs` / `find_mountpoint_and_fs`) 具体定义如下:

```

01
02 /// 表示一个挂载点: 包含挂载路径和对应的文件系统实例
03 struct MountPoint {
04     path: &'static str, // 挂载路径
05
06     (如 "/mnt/usb"), 生命周期为 'static 以简化管理
07
08     fs: alloc::sync::Arc<dyn VfsOps>, // 指向具体
09
10 文件系统的 Arc 引用, 支持动态分发
11
12 }

```



```

13
14
15 /// 根目录管理器，负责统一调度主文件系统与所有挂载点
16 pub struct RootDirectory {
17     main_fs: alloc::sync::Arc<dyn VfsOps>, // 主文件系
18
19     统（通常挂载在 "/"）
20
21     mounts: spin::RwLock<Vec<MountPoint>>, // 所有挂载
22
23     点列表，使用读写锁支持并发访问
24 }
25
26
27 impl RootDirectory {
28     /// 在指定路径挂载一个新的文件系统。
29     ///
30     /// - `path`：挂载点路径（必须是绝对路径，且不能为根 "/");
31     /// - `fs`：要挂载的文件系统实例。
32     ///
33     pub fn mount(&self, path: &'static str, fs:
34     alloc::sync::Arc<dyn VfsOps>) -> AxResult {
35         if path == "/" {
36             return ax_err!(InvalidInput, "cannot mount
37             root filesystem");
38         }
39
40         // 在主文件系统中创建挂载点目录（如 /mnt/usb）
41         self.main_fs.root_dir().create(path,
42         FileType::Dir)?;
43
44         // 通知新文件系统：你已被挂载到该路径
45         fs.mount(path,
46         self.main_fs.root_dir().lookup(path)?);
47
48         // 注册挂载点
49         self.mounts.write().push(MountPoint { path,
50         fs });
51
52         Ok(())
53     }
54 }
55
56

```

/// 在所有挂载点中执行最长前缀匹配，并对匹配到的文件系统调用回调函数。

```
///
/// - `path`：要查找的完整路径（如
"/mnt/usb/file.txt"）；

/// - `f`：回调函数，接收匹配到的文件系统和剩余子路径（如
fs 和 "file.txt"）。
```

```
///
/// 用途：用于 lookup、open、stat 等操作的路由分发。
fn lookup_mounted_fs<F, T>(&self, path: &str, f: F)
-> AxResult<T>
where
    F: FnOnce(alloc::sync::Arc<dyn VfsOps>, &str)
-> AxResult<T>
{
    // 对 self.mounts.read() 进行最长前缀匹配（例如
"/mnt/usb" 比 "/mnt" 更优）
    // ...existing code...
}
```

/// 查找给定路径所属的挂载点及其文件系统。

```
///
/// - `path`：输入路径（如 "/mnt/usb/data"）；

/// - 返回：匹配的挂载路径（如 "/mnt/usb"）和对应的文件
系统引用。
```

```
///
/// 内部会先对路径进行规范化（如处理 ".."、"."、多余 '/'），
再遍历挂载点列表寻找最佳匹配。
```

```
pub fn find_mountpoint_and_fs(&self, path: &str) ->
```

```

AxResult<(alloc::string::String, alloc::sync::Arc<dyn
VfsOps>)> {
    // canonicalize + iterate mounts to find best
    match, return (mount_path, fs)
    // ...existing code...
}
}

```

4.3 文件系统总结

Unfound OS 在 ArceOS 模块化内核的基础上，扩展虚拟文件系统（VFS）架构。通过统一 I/O 抽象模型（FileLike trait），将文件、目录及未来 I/O 对象纳入一致接口，奠定“一切皆可描述符”的基础；通过完整的 POSIX 目录支持，实现 `openat`、`mkdirat`、`unlinkat`、`getdents` 等关键系统调用；并通过动态多后端挂载机制，支持在任意路径（如 `/mnt/usb`）运行时挂载 FAT、EXT4、RAMFS 等多种文件系统，借助最长前缀匹配实现智能请求路由，兼顾灵活性与 POSIX 兼容性。

第 5 章 进程管理模块

5.1 进程管理模块框架概述

UnfoundOS 的进程管理模块基于 ArceOS 原有的组件化内核架构进行了深度扩展与重构。我们在保留 ArceOS 高效任务调度（``axTask``）特性的基础上，针对通用计算场景的需求，引入了符合 POSIX 标准的进程抽象层。该模块不仅增强了原有的任务模型，使其支持层级化管理（Process Tree），更引入了细粒度的并发控制（RwLock）、严格的资源隔离（Resource Isolation）以及丰富的进程间通信（IPC）机制。我们的工作重点在于解决原生 Unikernel 架构在面对复杂多进程应用时的局限性：缺乏隔离性、并发瓶颈以及生态兼容性不足。通过在 ArceOS 基础上引入 ``ProcessContext`` 抽象和全局 ``ProcessManager``，我们将系统升级为一个既保留了 Unikernel 轻量级特性，又支持多进程协作、具备自我清理能力的现代操作系统内核。

5.2 读写锁分离与并发架构实现

为了突破传统全局互斥锁带来的性能瓶颈，进程管理模块采用了读写锁分离架构。该架构的核心设计是将全局进程管理器 `process_manager` 从 `Mutex` 升级为 `Rwlock`，以此支持多读单写的并发模式；该架构具备三大关键特性，即支持高频查询操作（如 `getpid`、信号发送）的并行执行而无需阻塞、仅在进程创建与销毁

时进行写锁互斥以保证数据一致性、通过细粒度的锁策略显著提升了多核环境下的系统吞吐量。

```
01 pub static PROCESS_MANAGER: RwLock<ProcessManager> =
02   RwLock::new(ProcessManager::new());
03
04 pub struct ProcessManager {
05     /// 所有进程的映射表 (PID -> Process)
06     pub processes: BTreeMap<u32, Arc<Mutex<ProcessContext>>>,
07     /// 父子关系映射表 (PPID -> [Child PIDs])
08     pub parent_child_map: BTreeMap<u32, Vec<u32>>,
09     /// PID 分配器
10     next_pid: u32,
11 }
12
13 impl ProcessManager {
14     pub const fn new() -> Self {
15         Self {
16             processes: BTreeMap::new(),
17             parent_child_map: BTreeMap::new(),
18             next_pid: 1,
19         }
20     }
21
22     pub fn allocate_pid(&mut self) -> u32 {
23         let pid = self.next_pid;
24         self.next_pid = self.next_pid.wrapping_add(1);
25         if self.next_pid == 0 {
26             self.next_pid = 1;
27         }
28         pid
29     }
30
31     pub fn get_process(&self, pid: u32) ->
32     Option<Arc<Mutex<ProcessContext>>> {
33         self.processes.get(&pid).cloned()
34     }
35
36     pub fn remove_process(&mut self, pid: u32) {
37         self.processes.remove(&pid);
38         self.parent_child_map.remove(&pid);
39     }
40
41     pub fn get_children(&self, ppid: u32) -> Vec<u32> {
42         self.parent_child_map
```

```

43         .get(&ppid)
44         .cloned()
45         .unwrap_or_default()
    }
}

```

我们通过 `RwLock` 包装 `ProcessManager`，实现了读写分离。`processes` 使用 `BTreeMap` 存储，支持 $O(\log n)$ 的快速查找；`parent\_child\_map` 显式维护了进程树结构，使得查找子进程的操作极其高效。

5.3 进程生命周期与资源隔离实现

UnfoundOS 进程管理模块构建了健壮的生命周期管理机制，专门用于处理进程的创建、执行、等待与销毁，其核心支持涵盖多方面需求，包括孤儿进程重帕（Orphan Reparenting）以防止僵尸进程泄漏、非阻塞等待（WNOHANG）以支持异步监控、以及基于文件描述符表（FdTable）的资源隔离。

➤ 孤儿进程重帕机制：

```

01 pub fn sys_exit(exit_code: i32) -> ! {
02     {
03         let mut manager = PROCESS_MANAGER.write();
04         if let Some(children) = manager.parent_child_map.remove(&current_pid())
05     {
06         for child_pid in children {
07             // 更新子进程的父进程 ID (PPID) 为 1 (init 进程)
08             if let Some(child_proc_arc) = manager.get_process(child_pid) {
09                 let mut child_proc = child_proc_arc.lock();
10                 child_proc.ppid = 1;
11             }
12             // 将子进程添加到 init 进程 (PID=1) 的子进程列表中
13             manager.parent_child_map.entry(1).or_default().push(child_pid);
14         }
15     }
16 }
17 axtask::exit(exit_code)
    }
}

```

这是防止僵尸进程泄漏的最后一道防线。当父进程退出时，必须持有全局写锁（`write()`）来原子性地修改进程树，确保所有子进程都能被 Init 进程（PID 1）正确接管。

➤ 非阻塞等待实现：

```

01 pub fn sys_waitpid(pid: i32, wstatus: *mut i32, options: i32)
02 -> i32 {
03     // ...
04     loop {
05         // ... 查找已退出的子进程 ...

```

```

06
07     // 如果没有子进程退出, 且设置了 WNOHANG, 则立即返回
08     if options & WNOHANG != 0 {
09         return 0;
10     }
11
12     // 否则阻塞当前任务, 等待子进程状态变化信号
13     // ...
14 }
15 }

```

>文件描述符表虚拟化:

实现了从整数 FD 到内核对象的映射层。每个进程拥有独立的 `BTreeMap<i32, FdEntry>`。进程 A 的 FD 3 和进程 B 的 FD 3 互不干扰, 且支持 `dup`、`close` 等操作。

```

01 #[derive(Debug, Clone)]
02 pub struct ProcessContext {
03     // ...
04     /// Isolated File Descriptor Table
05     /// Maps integer FDs (0, 1, 2...) to global file objects
06     pub fd_table: BTreeMap<i32, FdEntry>,
07     pub next_fd: i32,
08     /// 最大文件描述符数量限制
09     pub max_fd: usize,
10     // ...
11 }
12 impl ProcessContext {
13     /// 添加新的 FD 条目
14     pub fn add_fd(&mut self, path: String, flags: u32) -> i32
15     {
16         if self.fd_table.len() >= self.max_fd {
17             return -1; // EMFILE
18         }
19         let fd = self.next_fd;
20         self.next_fd += 1;
21         self.fd_table.insert(fd, FdEntry {
22             fd,
23             path,
24             flags,
25         });
26         fd
27     }
28 }

```

`FdTable` 是实现资源隔离的关键。将整数 FD 映射到内核对象, `add\_fd` 方法还内置了 `max\_fd` 检查, 防止单个进程耗尽系统资源。

➤资源限制:

`RLIMIT\_NOFILE` 检查。在 `open` 或 `socket` 创建时, 检查当前进程的 FD 数量是否超过 `max\_fd` 限制, 防止单进程耗尽系统资源。

```
01 pub fn open_fd(path: String, flags: u32) -> i32 {
02     let current_task = current();
03     let current_pid =
04     (*current_task).task_ext().pid;
05
06     let manager = PROCESS_MANAGER.read();
07     if let Some(proc_arc) =
08     manager.get_process(current_pid) {
09         let mut proc = proc_arc.lock();
10         // 内部调用 add_fd 进行资源限制检查
11         proc.add_fd(path, flags)
12     } else {
13         -1
14     }
15 }
```

在每次打开文件时, 系统都会先获取当前进程的上下文, 并调用 `add\_fd` 进行配额检查。

5.4 进程间通信 (IPC) 系统实现

我们为每个进程构建了独立的资源视图, 实现了“零信任”的隔离模型。为

了支持复杂的进程协作，模块实现了多种 IPC 机制。

➤管道:

特性: 基于环形缓冲区 (Ring Buffer) 的字节流通信。

实现: 支持阻塞读写。当缓冲区满时写者阻塞，空时读者阻塞。

```
01 pub fn sys_pipe(pipefd: *mut i32) -> i32 {
02     // 创建新管道 (4KB 缓冲)
03
04     let pipe = Arc::new(PipeBuffer::new(4096));
05
06
07     // ... 注册到全局表并分配 FD ...
08
09     let read_fd = 1000 + pipe_id * 2;
10     let write_fd = 1000 + pipe_id * 2 + 1;
11
12     unsafe {
13         *pipefd = read_fd;
14         *pipefd.offset(1) = write_fd;
15     }
16     0
17 }
```

sys_pipe` 展示了如何在内核中创建一对关联的文件描述符。通过共享同一个 `PipeBuffer` (环形缓冲区)，读端和写端实现了数据的单向流动。

➤信号:

特性: 异步通知机制。

实现: 每个进程包含一个 `SignalQueue`。实现了 `sys\_kill` 发送信号，以及在中断/异常返回路径上的信号处理检查。

```
01 pub struct SignalQueue {
02     pending: Vec<Signal>,
03     handlers: BTreeMap<Signal,
04     Option<SignalHandler>>,
05 }
06
07 impl SignalQueue {
08     /// 发送信号: 将信号加入待处理队列
09
10     pub fn send(&mut self, sig: Signal) {
11         self.pending.push(sig);
12     }
13
14     /// 处理待处理的信号
15
16     pub fn process_pending(&mut self) {
17         while let Some(sig) = self.get_pending() {
18             if let Some(handler) =
```



```

19 self.get_handler(sig) {
20     handler(sig);
    }
    }
}

```

‘SignalQueue’ 是异步通知的核心数据结构。它将信号存储在队列中，等待进程在合适的时机进行处理。这种设计解耦了信号的发送与处理。

```

01 pub fn sys_kill(pid: u32, sig: i32) -> i32 {
02     // ... 信号转换 ...
03
04     let signal = match sig { /* ... */ };
05
06     // 获取目标进程（读锁优化）
07
08     let proc_arc = {
09         let manager = PROCESS_MANAGER.read();
10         manager.get_process(pid)
11     };
12
13     if let Some(proc_arc) = proc_arc {
14         let mut proc = proc_arc.lock();
15
16         // 向目标进程发送信号
17         proc.send_signal(signal);
18         0
19     } else {
20         AxError::NoProcess.to_i32()
21     }
22 }

```

‘sys_kill’ 展示了进程间交互的标准模式：查找目标进程 -> 获取锁 -> 修改状态。这里利用了 ‘PROCESS_MANAGER’ 的读锁来快速定位目标。

➤消息队列：

特性：System V 风格的报文传递。

实现：支持消息类型过滤。接收者可以指定接收特定类型的消息（如只处理高优先级消息），比管道更灵活。

```

01 pub fn sys_msgsnd(msqid: i32, msgp: *const u8, msgsz:
02     usize, msgflg: i32) -> i32 {
03     // ...
04
05     // 检查队列是否已满
06
07     if q.current_bytes + msgsz > q.max_bytes {
08         if msgflg & IPC_NOWAIT != 0 {
09             return AxError::ResourceBusy.to_i32(); //
10         }
11         EAGAIN
12     }
13 }

```

```

10         }
11         // 阻塞等待：当前任务挂起，直到有空间
12
13         q.send_wait.wait();
14         continue;
15     }
16
17     // 入队并唤醒接收者
18
19     q.msgs.push_back(Message { mtype, text });
20     q.recv_wait.notify_one(false);
21     return 0;
22 }

```

实现了“生产者-消费者”模型。通过 `WaitQueue` (`send\_wait`) 实现阻塞等待，通过 `notify\_one` 唤醒对端进程。

5.5 进程管理模块总结

本章详细阐述了 UnfoundOS 在进程管理模块上的深度重构与创新工作。针对原生 Unikernel 架构在多进程支持上的不足，我们从数据结构、并发控制、生命周期管理到通信机制进行了全方位的升级。

- 架构现代化：通过引入 `ProcessContext` 和全局 `ProcessManager`，我们成功构建了符合 POSIX 标准的进程抽象层，实现了从单一任务模型到层级化进程树的跨越。
- 并发性能优化：采用 RwLock 读写分离策略重构了核心管理器，有效解决了高并发场景下的锁竞争问题，显著提升了系统在多核环境下的扩展性。
- 健壮性保障：实现了孤儿进程重帕和僵尸进程回收机制，彻底杜绝了资源泄漏的风险，确保了系统长期运行的稳定性。
- 功能完备性：构建了包含信号、管道、消息队列在内的完整 IPC 子系统，并实现了文件描述符级别的资源隔离，为上层复杂应用提供了丰富的系统调用支持。

第 6 章总结与展望

6.1 项目总结

Unfound OS 基于 ArceOS 模块化内核框架，以 Rust 语言的内存安全特性为基石，完成了从实验性 Unikernel 到具备完整宏内核核心能力的操作系统原型演进。项目聚焦内存管理、文件系统、进程管理三大核心模块，通过系统性设计与优化，显著提升了内核的功能性、稳定性、兼容性与并发能力，为后续扩展奠定了坚实基础。

在内存管理模块，项目创新构建了多分配器框架与全维度测试体系：通过

PageAllocator trait 抽象统一接口，实现了 Buddy、Bitmap、Hybrid 三种分配算法的无缝集成与运行时动态切换，支持无需重新编译内核即可灵活选择分配策略；配套的测试框架不仅覆盖基础性能、碎片率、稳定性、时间维度及内核兼容性五大测试场景，还具备可视化分析与报告生成能力——可自动生成 Markdown 格式测试报告，整合性能指标、碎片分析、时间序列数据，支持多分配器对比视图，既保障了分配器的可靠性，也为技术评估与教学演示提供了直观、可复用的实验成果载体，有效弥合理论教学与工程实践之间的鸿沟。

在文件系统模块，项目针对 ArceOS 原有 VFS 的功能局限，提出了三大核心改进：通过扩展 VfsOps 与 VfsNodeOps trait 构建统一 I/O 抽象模型，支持更丰富的元数据管理与设备适配；完整实现 POSIX 标准中的 *at 系列系统调用与目录操作语义，兼容 GNU coreutils 等传统用户态程序；创新设计动态多后端挂载机制，支持在任意路径运行时挂载多种文件系统，并通过最长前缀匹配实现嵌套挂载的智能请求路由，大幅提升了文件系统的灵活性与扩展性。

在进程管理模块，项目基于 ArceOS 原有 axtask 组件进行深度重构与扩展，构建了符合 POSIX 标准的进程抽象层：通过 ProcessContext 抽象与全局 ProcessManager 实现层级化进程树管理，解决了原生 Unikernel 缺乏隔离性与生态兼容性的问题；采用 RwLock 读写分离架构优化并发控制，支持多读单写模式，显著提升多核环境下的系统吞吐量；实现了健壮的进程生命周期管理，包括孤儿进程重父、非阻塞等待 (WNOHANG) 机制，杜绝僵尸进程泄漏；通过独立文件描述符表 (FdTable) 与资源限制 (RLIMIT_NOFILE) 实现细粒度资源隔离；构建了完整的 IPC 子系统，涵盖管道、信号、消息队列三种核心通信机制，支持复杂进程协作场景。

6.2 核心价值

项目的核心价值体现在技术创新与实践意义两个层面：技术上，延续并深化了 ArceOS 的模块化设计理念，通过 Rust trait 抽象、动态分发与读写分离等技术，实现了核心模块的高内聚、低耦合，为操作系统内核的灵活扩展与组件复用提供了可复用的设计范式——进程管理模块的并发架构优化、文件系统的动态挂载机制、内存管理的多分配器切换与可视化测试报告能力，共同构成了“模块化 + 可扩展 + 可观测”的内核设计典范；实践上，三大核心模块的协同演进使 Unfound OS 具备了运行复杂多进程应用的能力，同时内存管理的可视化测试框架、文件系统与进程管理的 POSIX 兼容性设计，使其可作为操作系统课程的实验平台，既能输出量化可比的实验数据，又能通过直观报告降低教学演示与成果展示的门槛。

6.3 现存局限

尽管项目已实现核心功能的突破，但仍存在以下可优化空间：

内存管理模块：Hybrid 分配器的策略选择逻辑可进一步优化，尚未实现基于系统负载与分配请求特征的自适应调整；当前可视化报告虽支持 Markdown 格式输出，但缺乏交互式图表（如折线图、柱状图）集成，数据趋势展示不够直观，

且未支持报告的导出与共享功能。

文件系统模块：尚未支持文件权限控制、硬链接 / 软链接等 POSIX 高级特性，多种文件系统后端的性能适配不足，高并发 I/O 场景下的表现仍有提升空间。

进程管理模块：IPC 机制缺乏共享内存支持，难以满足高吞吐量数据传输场景需求；进程调度策略仍沿用原有 axtask 的基础调度算法，未针对多核负载均衡、实时性需求进行优化；信号机制仅支持基础异步通知，未实现信号掩码、信号等待 (sigwait) 等完整 POSIX 语义。

内核整体：功耗管理、中断优化及更多硬件平台的适配工作尚未开展，限制了系统的部署范围；缺乏完整的用户态权限管理模型（如 UID/GID），难以支持多用户场景。

6.4 未来展望

未来，Unfound OS 将围绕 “功能完善、性能优化、生态扩展” 三大方向持续演进，重点推进以下工作：

6.4.1 功能完善

内存管理：优化 Hybrid 分配器的自适应策略，引入基于机器学习的分配算法选择机制；升级可视化测试框架，新增交互式图表（折线图、柱状图、热力图）集成，支持测试报告导出为 PDF/HTML 格式，新增报告共享与版本管理功能；扩展测试场景，新增极端负载下的分配器稳定性测试。

文件系统：补充权限管理、链接机制、磁盘配额等 POSIX 完整特性；支持日志型文件系统（如 EXT4）的崩溃恢复功能；新增网络文件系统（NFS）挂载支持。

进程管理：新增共享内存（Shared Memory）IPC 机制，满足高吞吐数据传输需求；完善信号机制的 POSIX 兼容性，支持信号掩码、sigwait 等特性；引入 UID/GID 权限模型，支持多用户隔离与访问控制；新增进程管理相关系统调用（如 fork 优化、exec 系列完整实现）。

模块扩展：计划新增网络协议栈（TCP/IP）、设备驱动框架增强（USB/PCIe 支持）、虚拟内存管理（地址空间隔离优化）等核心模块，构建功能完备的操作系统内核。

6.4.2 性能优化

内存管理：针对多线程竞争问题引入无锁设计，优化 SwitchingAllocator 的切换开销；提升分配算法的局部性，减少缓存失效；优化可视化报告的数据处理逻辑，降低大体积测试数据的报告生成耗时。

文件系统：对路径解析与 I/O 操作进行性能调优，引入页缓存（Page Cache）机制减少磁盘访问延迟；优化动态挂载的最长前缀匹配算法，提升路径路由效率。

进程管理：优化进程调度策略，引入 CFS（完全公平调度）或实时调度算法，支持多核负载均衡；减少进程创建与切换的开销，通过内核线程复用提升系统响应速度；优化 IPC 机制的阻塞 / 唤醒逻辑，降低上下文切换成本。

整体优化：借助 Rust 的异步编程特性，实现 I/O 操作、进程调度的异步化，提升系统的并发处理能力；针对嵌入式场景进行内核裁剪与轻量化优化。

6.4.3 生态扩展

硬件适配: 完善内核的硬件适配层, 支持更多嵌入式设备 (如 Raspberry Pi 系列)、服务器硬件平台及虚拟化环境 (KVM/QEMU 优化)。

应用生态: 构建更丰富的用户态工具链与应用生态, 适配更多 GNU 工具集与开源应用; 提供完整的开发文档与示例代码, 降低应用移植门槛; 将内存管理的可视化测试框架开源, 支持第三方分配器的接入与测试。

社区共建: 开放多分配器框架、文件系统抽象与进程管理模块的接口文档, 鼓励社区参与组件开发与扩展; 建立内核测试与验证体系, 支持第三方组件的兼容性测试; 搭建社区报告分享平台, 促进实验成果与技术经验的交流。

最终, Unfound OS 旨在成为一个兼具稳定性、灵活性与可扩展性的通用操作系统内核原型, 既可为学术研究与教学提供实验平台, 也能为嵌入式系统、云原生、边缘计算等场景提供高性能、内存安全的内核解决方案。

第 7 章致谢

本项目基于开源社区的优秀成果进行开发, 部分代码和模块参考或直接使用了他人开源项目。

本项目基于 ArceOS 内核进行二次开发, ArceOS 是一个由社区驱动、采用 Rust 语言编写的实验性模块化操作系统。ArceOS 受 Unikraft 等项目启发, 支持多种架构和平台, 具备丰富的内核功能。

主要依赖与引用的开源项目

- ArceOS
- axconfig-gen (内核配置工具)
- arceos-crates
- musl (C 运行时工具链)
- 其他依赖详见 Cargo.toml

本项目遵循多重开源协议 (GPL-3.0-or-later OR Apache-2.0 OR MulanPSL-2.0), 具体以各模块 LICENSE 文件为准。如需商用或二次分发, 请务必遵守相关开源协议要求, 并保留原作者署名与许可证声明。

感谢杭州电子科技大学杨浩老师和刘真老师的指导!